Sneha Patra **D20A** **43**

Blockchain Experiment 2

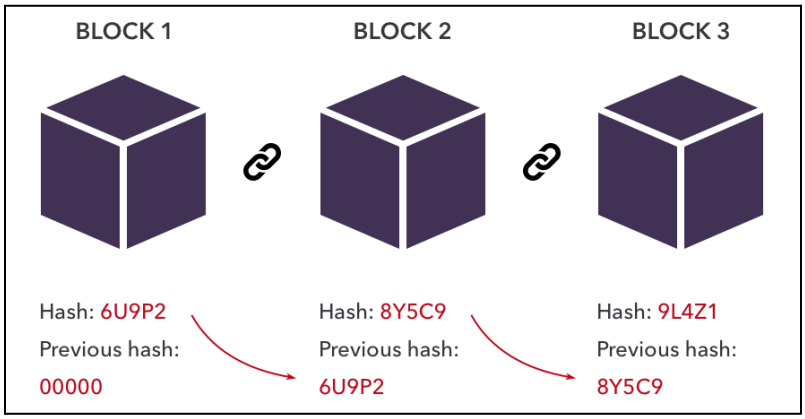
AIM: Create a Blockchain using Python

THEORY:

Q1: What is a Blockchain?

A Blockchain is a distributed and decentralized digital ledger that records transactions across multiple computers in a secure and transparent manner. Each record is stored in a block, and these blocks are linked together using cryptographic hash values, forming a chain.

Once data is added to a block and confirmed, it cannot be altered or deleted, which makes blockchain highly secure and tamper-resistant. Blockchain technology removes the need for a central authority and allows participants in the network to trust the system through cryptographic techniques and consensus mechanisms.



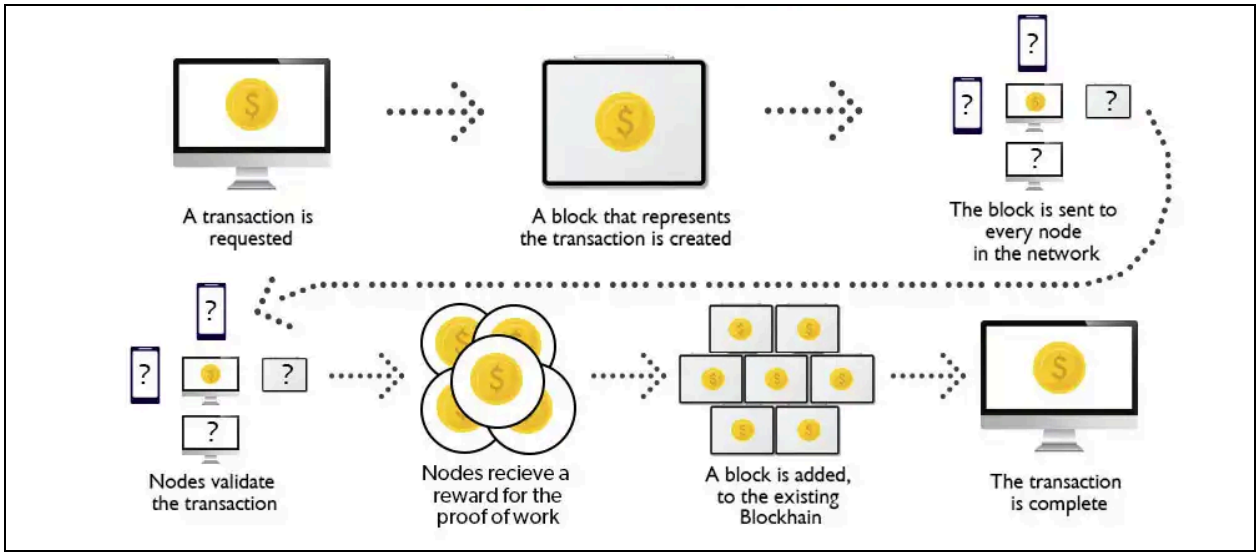
Q2: Process of Mining

Mining is the process of adding new blocks to the blockchain. It involves validating transactions and solving a cryptographic puzzle using computational power.

Steps in the Mining Process:

a. Transactions are collected and grouped into a block.
b. Miners calculate the hash of the block data along with a nonce.
c. The nonce is repeatedly changed until the hash meets the network's difficulty condition (e.g., leading zeros).
d. The first miner to find the valid hash broadcasts the block to the network.
e. Other nodes verify the solution.
f. Once verified, the block is added to the blockchain.

Mining ensures network security, prevents double spending, and maintains consensus among participants.



Q3: How to check the validity of blocks in a Blockchain

The validity of blocks in a blockchain is checked to ensure that the data has not been tampered with and that all blocks follow the network rules.

Block Validation Process:

- **Previous Hash Verification:**
  Each block stores the hash of the previous block. If this hash does not match the actual hash of the previous block, the chain is invalid.
- **Proof-of-Work Verification:**
  The block's proof value is checked to confirm that it satisfies the required difficulty condition.
- **Hash Integrity Check:**
  The block data is re-hashed to ensure the hash value remains unchanged.
- **Chain Continuity Check:**
  Blocks are validated sequentially from the genesis block to the latest block.

If all these conditions are satisfied, the blockchain is considered valid and secure.

<u>TASKS PERFORMED:</u>

Task: Implementation of a Simple Blockchain with Proof-of-Work and Flask API

```
# Importing libraries
import datetime          # To generate timestamps for blocks
import hashlib           # To generate SHA-256 hashes
import json              # To convert block data into JSON format
from flask import Flask, jsonify   # To create REST API endpoints

# Part 1 - Building the Blockchain
class Blockchain:

    def __init__(self):
```

```python
        # List to store the blockchain
        self.chain = []

        # Create the genesis block
        self.create_block(proof=1, previous_hash='0')

    def create_block(self, proof, previous_hash):
        """
        Create a new block and add it to the blockchain
        """
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash
        }

        self.chain.append(block)
        return block

    def get_previous_block(self):
        """
        Return the last block in the chain
        """
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        """
        Proof of Work Algorithm:
        Find a number such that the hash of
        (new_proof^2 - previous_proof^2)
        starts with '0000'
        """
        new_proof = 1
        check_proof = False

        while not check_proof:
            hash_operation = hashlib.sha256(
                str(new_proof**2 - previous_proof**2).encode()
            ).hexdigest()
```

```
        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1

    return new_proof

def hash(self, block):
    """
    Create a SHA-256 hash of a block
    """
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    """
    Check whether the blockchain is valid
    """
    previous_block = chain[0]
    block_index = 1

    while block_index < len(chain):
        block = chain[block_index]

        # Check previous hash
        if block['previous_hash'] != self.hash(previous_block):
            return False

        # Check proof of work
        previous_proof = previous_block['proof']
        proof = block['proof']

        hash_operation = hashlib.sha256(
            str(proof**2 - previous_proof**2).encode()
        ).hexdigest()

        if hash_operation[:4] != '0000':
            return False
```

```
            previous_block = block
            block_index += 1

        return True


# Part 2 - Mining the Blockchain (Flask API)
# Create Flask app
app = Flask(__name__)

# Create Blockchain object
blockchain = Blockchain()

# API Endpoint - Mine a new block
@app.route('/mine_block', methods=['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']

    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)

    block = blockchain.create_block(proof, previous_hash)

    response = {
        'message': 'Congratulations Sneha, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash']
    }

    return jsonify(response), 200


# API Endpoint - Get full blockchain
@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
```

```
        'length': len(blockchain.chain)
    }
    return jsonify(response), 200


# API Endpoint - Check blockchain validity
@app.route('/is_valid', methods=['GET'])
def is_valid():
    valid = blockchain.is_chain_valid(blockchain.chain)

    if valid:
        response = {'message': 'Blockchain is valid, Sneha.'}
    else:
        response = {'message': 'Blockchain is NOT valid, Sneha.'}

    return jsonify(response), 200


# Run the Flask Application
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Not secure 192.168.36.231:5000/mine_block

Pretty-print


```
{
  "index": 2,
  "message": "Congratulations Sneha! You mined a block.",
  "previous_hash": "883091ca5b8f8f55a5ca920921a1de66b40438a47460dffa33f5e2d9d4e3b133",
  "proof": 533,
  "timestamp": "2026-02-06 12:49:14.864973"
}
```


Not secure 192.168.36.231:5000/get_chain

Pretty-print


```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-02-06 12:48:29.199214"
    },
    {
      "index": 2,
      "previous_hash": "883091ca5b8f8f55a5ca920921a1de66b40438a47460dffa33f5e2d9d4e3b133",
      "proof": 533,
      "timestamp": "2026-02-06 12:49:14.864973"
    }
  ],
  "length": 2
}
```


Not secure 192.168.36.231:5000/is_valid

Pretty-print


```
{"message":"Blockchain is valid."}
```

Not secure 192.168.36.231:5000/is_valid

Pretty-print


```
{
  "message": "Blockchain is valid."
}
```


Windows PowerShell | Windows PowerShell

```
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\STUDENT> py -3.11 --version
[ERROR] No runtime installed that matches 3.11. Try running "py install 3.11".
PS C:\Users\STUDENT> py install 3.11
Installing Python 3.11.9.
Downloading: ..................................................................
Extracting: ...................................................................
Python 3.11.9 will be launched by python3.11[-64].exe
PS C:\Users\STUDENT> py -3.11 -m pip install flask==2.2.5
Collecting flask==2.2.5
  Using cached Flask-2.2.5-py3-none-any.whl.metadata (3.9 kB)
Collecting Werkzeug>=2.2.2 (from flask==2.2.5)
  Using cached werkzeug-3.1.5-py3-none-any.whl.metadata (4.0 kB)
Collecting Jinja2>=3.0 (from flask==2.2.5)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting itsdangerous>=2.0 (from flask==2.2.5)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting click>=8.0 (from flask==2.2.5)
  Using cached click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting colorama (from click>=8.0->flask==2.2.5)
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting MarkupSafe>=2.0 (from Jinja2>=3.0->flask==2.2.5)
  Downloading markupsafe-3.0.3-cp311-cp311-win_amd64.whl.metadata (2.8 kB)
Using cached Flask-2.2.5-py3-none-any.whl (101 kB)
Using cached click-8.3.1-py3-none-any.whl (108 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached werkzeug-3.1.5-py3-none-any.whl (225 kB)
Downloading markupsafe-3.0.3-cp311-cp311-win_amd64.whl (15 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: MarkupSafe, itsdangerous, colorama, Werkzeug, Jinja2, click, flask
  WARNING: The script flask.exe is installed in 'C:\Users\STUDENT\AppData\Local\Python\pythoncore-3.11-64\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed Jinja2-3.1.6 MarkupSafe-3.0.3 Werkzeug-3.1.5 click-8.3.1 colorama-0.4.6 flask-2.2.5 itsdangerous-2.2.0

[notice] A new release of pip is available: 24.0 -> 26.0.1
[notice] To update, run: C:\Users\STUDENT\AppData\Local\Python\pythoncore-3.11-64\python.exe -m pip install --upgrade pip
PS C:\Users\STUDENT> cd C:\Users\STUDENT\Desktop\D20A_43
PS C:\Users\STUDENT\Desktop\D20A_43> py -3.11 blockchain.py
 * Serving Flask app 'blockchain'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.36.231:5000
Press CTRL+C to quit
127.0.0.1 - - [06/Feb/2026 12:43:22] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [06/Feb/2026 12:43:22] "GET /favicon.ico HTTP/1.1" 404 -
192.168.36.231 - - [06/Feb/2026 12:43:51] "GET / HTTP/1.1" 404 -
192.168.36.231 - - [06/Feb/2026 12:43:51] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [06/Feb/2026 12:45:00] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [06/Feb/2026 12:47:49] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [06/Feb/2026 12:47:51] "GET /mine_block HTTP/1.1" 200 -
PS C:\Users\STUDENT\Desktop\D20A_43> py -3.11 blockchain.py
 * Serving Flask app 'blockchain'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.36.231:5000
Press CTRL+C to quit
192.168.36.231 - - [06/Feb/2026 12:48:37] "GET / HTTP/1.1" 404 -
192.168.36.231 - - [06/Feb/2026 12:48:52] "GET /block_mine HTTP/1.1" 404 -
192.168.36.231 - - [06/Feb/2026 12:48:54] "GET /block_mine HTTP/1.1" 404 -
192.168.36.231 - - [06/Feb/2026 12:49:14] "GET /mine_block HTTP/1.1" 200 -
192.168.36.231 - - [06/Feb/2026 12:49:55] "GET /get_chain HTTP/1.1" 200 -
```

CONCLUSION:

In this experiment, a simple blockchain was successfully implemented using Python and Flask. The system allows block mining, viewing the complete blockchain, and checking its validity. Proof-of-Work was used to make block creation secure, while cryptographic hashing ensured that the data stored in blocks remains unchanged. This implementation helps in understanding the basic concepts of blockchain such as immutability, consensus, and decentralization, and provides a good foundation for learning advanced blockchain applications.